CS3103 Group 15 Assignment 4 Miniproject Report

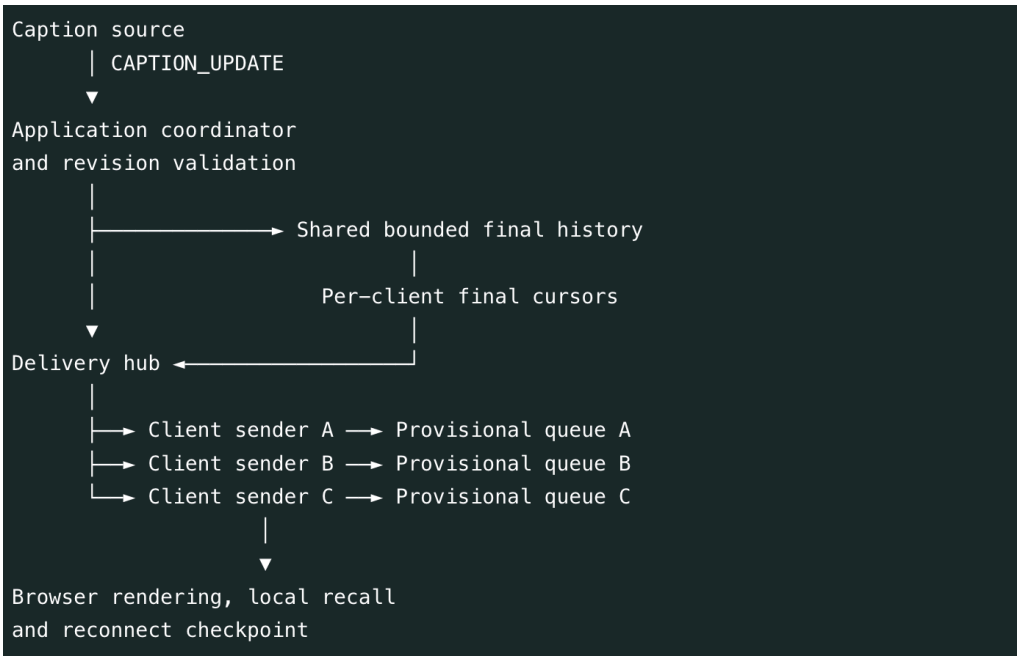
Alqasim Elyas Zuhair A (A0276062E), Ng Wei Sheng Vincent (A0282202U), Ong Li Hong (A0276098M), Yong Kotaro (A0258311L)

1. Introduction

LiveRecall uses revision-aware state validation, shared bounded final history and independent per-client senders. During congestion, obsolete provisional revisions are coalesced while finalized captions remain available through per-client history cursors.

2. Architecture and Protocol

The main responsibilities and data flow are shown below:



The application coordinator identifies source and client connections and controls access to shared state. The revision state validates caption fields and rejects invalid, duplicate, stale, or already-finalized updates. Accepted finals are stored once in the bounded shared history. Each client keeps a cursor pointing to its next required final to avoid a separate final queue for every client.

The delivery hub publishes accepted updates to independent client sessions without waiting for network sends. Each client session has its own bounded provisional queue and sender task. Therefore, a slow client does not block the source or other clients. The browser renders revisions, stores recent finals for local recall, and remembers its latest finalized caption for reconnection.

Communication uses JSON messages over WebSockets:

Message	Purpose
SOURCE_HELLO	Establish the caption-source role.
CLIENT_HELLO	Provide the session, finalized checkpoint, and requested history window.
CAPTION_UPDATE	Carry the segment, revision, timestamp, final flag, and caption text.
HISTORY	Return recent missing finalized captions.

The compatible startup command is: python3 solution_server.py --port 8765

The server uses Python's standard asynchronous features and the lightweight websockets package. It does not require a database, cloud service, credentials or external speech-recognition model.

3. Revision Correctness, History, and Recovery

A caption usually arrives as provisional revisions followed by a final revision, all sharing a segment ID. Our design associates updates by segment ID and uses revision numbers to determine whether they replace previously accepted versions. The server's CaptionState class validates message fields, tracks the latest provisional revision, and rejects older or equal revisions. A highest-finalized-segment variable protects completed captions from subsequent updates, relying on the source completing segments sequentially. The browser independently checks revisions and finalization status, updating the segment's existing DOM element rather than appending duplicate lines.

The server's FinalHistory class stores finalized captions for 70 seconds after storage, with limits of 20,000 entries and 32 MiB of serialized payload. A deque supports removal of expired entries from oldest to newest, while a sequence-indexed dictionary provides direct lookup for independent client senders. Both structures reference the same entries. Each final receives an increasing internal sequence number, distinct in purpose from its protocol segment ID. Each client maintains a cursor identifying its next final, if that final has expired, the server explicitly closes the connection rather than silently skipping it.

The browser separately stores 70 seconds of finals in a segment-keyed map. Its live display shows a 15-second caption window with autoscroll. The "What did I miss?" feature gives the client the ability to retrieve 15-, 30-, or 60-seconds worth of captions. Recall uses locally stored finals without additional server requests.

Upon reconnection, the browser reports its latest finalized segment and revision in CLIENT_HELLO. The server selects recent finals within the requested window, excluding those already reported. History selection, recording the next final-history sequence to deliver, and client registration occur together under the application lock. This prevents source updates

from interleaving with client attachment and creating a gap between recovered history and live delivery. The sender transmits HISTORY before subsequent live updates.

Recovered finals pass through the browser's existing revision checks, preventing duplicate display. The latest accepted finalized segment and revision form its recovery checkpoint, telling the server what the browser already holds. New clients set both values to zero to request recent context. Automatic reconnection uses exponential backoff with jitter to spread retry attempts across clients.

4. Client Isolation and Semantic Backpressure

Initially, the baseline server waits for all client sends of the current caption to finish before processing the next caption in its buffer. This means that if one or more clients are blocked, the server's source handler delays and stays waiting rather than processing incoming captions.

To deal with this, the idea is to have an independent sender for each server-client connection. These senders run within the same server process but if a client is blocked, it would not stop delivery to other clients. The independent senders are represented by ClientSession objects that manage their own pending provisional queue, final-history bookmark and sending task.

The source handler can therefore process captions without waiting for all client sends to be complete. Accepted finals are first stored in shared FinalHistory. Both accepted finals and provisionals are then published through the DeliveryHub.publish(). Provisionals enter each session's queue, while finals only notify and wake the sender.

Each ClientSession runs a sender_loop() that reads provisionals from its queue. As for final captions, the sender uses its final-history bookmark as well as segment numbers to decide which final caption in the shared FinalHistory to send in order to preserve order. The reason we decided not to enqueue final captions as well is because since the shared final history already exists, we can make use of it to avoid storing essentially a duplicate backlog of final captions for every sender queue, thus saving memory.

Although independent senders prevent clients from blocking one another, the issue of slow clients still needs to be addressed as it may result in accumulation of captions in their sender queue. Hence, every time a caption is accepted to be queued, there will be a check for congestion in this queue using ClientSession.update_pressure() based on a multitude of variables like queue size, send duration, pending finals, etc.

However, even if the sender's queue is not congested, it does not prove that the client's network buffer is keeping up. There may be a situation where the sender's queue is constantly sending which results in no congestion detected at the sender, but captions accumulating in the network buffer at the client. Hence, to detect slow progress at the client's side as well , ClientSession.delivery_checkpoint() will periodically send a ping to the client after a bounded batch of messages. If the round-trip time for the returning pong from the client to

the sender is above a certain threshold, it suggests there is slow progress along the connection.

Therefore, when congestion is detected from either the sender or client, the sender will compact() the queue by removing provisionals whose segments are already finalised and keep only the latest revision of an unfinished segment. The sender_loop() also briefly defers sending of remaining provisionals so that a newer provisional can replace an older queued revision, or an arriving final can make those provisionals unnecessary. Final captions continue to be retrieved from shared history using the sender's final-history bookmark.

5. Experimental Methodology, Results, and Analysis

Stage 5 implements client isolation through independent per-client delivery queues, while Stage 6 adds semantic backpressure through congestion detection and provisional-update coalescing. This evaluation compares latency, freshness, final completeness, and revision coverage across client-load (T1–T4) and network-impairment (T5) benchmarks. All reported results are medians of three runs, with full results in Appendix A.

T1–T2: Normal Operation. Both stages achieve 100% freshness, final completeness, and revision coverage. Stage 6 increases P95 latency by 72.9% in T1 (12.02 to 20.79 ms) and 26.4% in T2 (53.49 to 67.63 ms) due to additional congestion monitoring.

T3: Slow Clients. With 10% slow readers, Stage 6 reduces overall P95 latency by 96.88% (2315.49 to 72.32 ms) and P99 by 77.32%. Freshness improves from 94.65% to 97.16%, fresh-final delivery from 90.00% to 94.61%, and final completeness from 95.13% to 100%. This is due to Stage 6 detecting congestion and coalescing obsolete provisional updates, reducing the work that slow clients must consume. Finals remain available through shared history and each client's delivery bookmark, allowing all clients to receive all finals.

T4: Reconnection. Both stages achieve 100% final completeness and live-update freshness, with 98.25% revision coverage. Stage 6 increases live P95 from 54.11 to 60.32 ms, while recovery P50/P95 improves from 67.19/82.05 to 60.54/76.41 ms. Stage 6's lower median recovery latency reflects reduced pending provisional work, allowing initial HISTORY responses to be scheduled sooner. This differs from live delivery, where congestion monitoring adds overhead.

Network-Impairment Benchmarks (T5)

Bandwidth Limitation (2 Mbit/s). Stage 6 reduces P95 by 19.2% (89.02 to 71.91 ms) but increases P99 by 19.3%. Both stages maintain 100% freshness, completeness, and coverage, indicating mixed latency effects without delivery-quality improvement.

Network Delay (100 ms). Stage 6 increases P95 by 11.4% (272.23 to 303.22 ms) while preserving full delivery. RTT-dependent feedback operations may introduce additional waiting, although direct timing measurements are needed to confirm this.

Delay and Loss. Stage 6 preserves 100% final completeness, but its other delivery metrics deteriorate. Freshness falls from 88.47% to 81.71%, fresh-final delivery from 89.65% to 81.76%, and revision coverage from 99.99% to 58.12%. P50, P95, and P99 also increase. The results suggest that congestion handling suppresses considerable provisional work without overcoming the underlying network delays.

Packet Loss (3%). Stage 6 reduces P50 by 21.8% (41.17 to 32.20 ms) but increases P95 by 25.3%. Revision coverage drops from 99.93% to 74.59%, while final completeness remains 100%. Temporary transport stalls may trigger unnecessary suppression without improving tail latency.

Reordering. Stage 6 increases P95 by 27.7% (88.21 to 112.65 ms), despite both stages maintaining full freshness, completeness, and coverage. This indicates additional latency due to the congestion monitoring.

6. Limitations and Future Work

Overall, semantic backpressure is effective as a targeted congestion-management mechanism rather than a general latency optimization. Under network impairments, congestion monitoring can mistake temporary transport stalls for sustained congestion, unnecessarily suppressing provisional revisions while failing to improve tail latency. Future improvements should distinguish it, reduce unnecessary feedback overhead, and validate final completeness.

Appendix A: Mininet Benchmark Results

All reported values are medians of three runs. Caption-age percentiles measure live-caption delivery latency and are distinct from reconnection recovery latency.

Table A1. Mininet Client-Load Benchmarks (T1–T4)

Test	Stage	Caption-age P50 (ms)	Caption-age P95 (ms)	Caption-age P99 (ms)	Fresh (%)	Fresh Finals (%)	Complete (%)	Coverage (%)
T1	5	9.66	12.02	13.19	100.00	100.00	100.00	100.00
T1	6	12.80	20.79	24.34	100.00	100.00	100.00	100.00
T2	5	39.01	53.49	56.07	100.00	100.00	100.00	100.00
T2	6	40.43	67.63	79.93	100.00	100.00	100.00	100.00
T3	5	38.75	2315.49	24208.54	94.65	90.00	95.13	95.18
T3	6	37.61	72.32	5491.34	97.16	94.61	100.00	94.30
T4	5	38.49	54.11	58.15	100.00	98.42	100.00	98.25
T4	6	38.44	60.32	75.08	100.00	98.42	100.00	98.25

Benchmark configurations: T1 uses 20 clients over 20 seconds. T2 uses 100 clients over 40 seconds. T3 uses 100 clients over 60 seconds, including 10% slow readers with a 500 ms delay after each update. T4 uses 100 clients over 60 seconds, with 20% experiencing temporary disconnections.

Table A2. Mininet Network-Impairment Benchmarks (T5)

Profile	Stage	Caption-age P50 (ms)	Caption-age P95 (ms)	Caption-age P99 (ms)	Fresh (%)	Fresh Finals (%)	Complete (%)	Coverage (%)
2 Mbit/s	5	73.62	89.02	93.67	100.00	100.00	100.00	100.00
2 Mbit/s	6	44.53	71.91	111.74	100.00	100.00	100.00	100.00
100 ms delay	5	227.76	272.23	290.47	100.00	100.00	100.00	100.00
100 ms delay	6	237.55	303.22	335.29	100.00	100.00	100.00	100.00

Profile	Stage	Caption-age P50 (ms)	Caption-age P95 (ms)	Caption-age P99 (ms)	Fresh (%)	Fresh Finals (%)	Complete (%)	Coverage (%)
Delay + loss	5	277.81	799.48	1168.05	88.47	89.65	100.00	99.99
Delay + loss	6	294.23	961.98	1249.81	81.71	81.76	100.00	58.12
3% loss	5	41.17	108.89	261.83	99.99	99.98	100.00	99.93
3% loss	6	32.20	136.44	252.82	99.98	99.94	100.00	74.59
Reordering	5	72.86	88.21	91.44	100.00	100.00	100.00	100.00
Reordering	6	87.19	112.65	137.86	100.00	100.00	100.00	100.00

Benchmark configuration: T5 uses 100 clients over 40 seconds, evaluated separately under five network-impairment profiles.